

Signals Project Report

Ain Shams University
Faculty of Engineering

Under the Supervision of
Prof. Ahmed Othman

Team Members

Name	Code	Section
Moaaz Mostafa Ibrahim Ahmed	*2400862*	*7*
Mohamed Ibrahim Abdalla Ibrahim Abdelfattah	*2401339*	*7*
Mohammad Abduljalil Mohammad Al Saeid	*2400877*	*7*
Mohammad Hany Ahmed Ali	*2400615*	*7*
Moaz Wael Mahmoud Mohamed	*2400733*	*7*
Mena Elia Rady Ghargoury	*2400609*	*7*

0 Tools Used

To implement this project in python multiple libraries must be imported for audio file reading/writing, FFT, array manipulation, and graphing. For audio files, `soundfile` was used as "`sf`" in the code. `Numpy` was used (as "`np`" and "`FT`") for FFT, IFFT, and array manipulation. Along with `matplotlib` as "`plt`" for plotting.

```
import numpy as np
import numpy.fft as FT
import matplotlib.pyplot as plt
import soundfile as sf
```

1 Time Domain Signal

A few lines of code can be used to extract the audio signal and plot it provided the audio file is in `.wav` format. After reading the file, we must change the shape (dimensions) of the array to a 1D array representing a mono audio signal.

```
x_t, f_s = sf.read('Moaaz.wav')

# Convert to mono
x_t = x_t.mean(axis=1) if x_t.ndim > 1 else x_t
# Define time domain
t = np.linspace(0, len(x_t) / f_s, num=len(x_t))
```

We can then plot[1] the signal as shown in figure (1).

```
time_fig, time_ax = plt.subplots()

time_ax.plot(t, x_t, color='k')
time_ax.set_xlabel('Time [s]')
time_ax.set_ylabel('Amplitude')
time_ax.set_title('Audio Signal')

time_fig.savefig('Moaaz_Signal.png')
plt.close(time_fig)
```

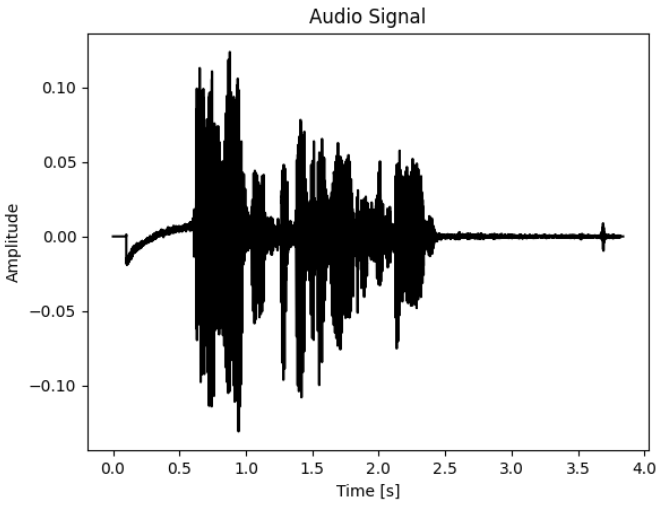


Figure 1: Time Domain Signal

[1] This plotting code will be used many times throughout the report. We will only show it this time but any plot in the report was generated similarly.

Lastly, we need to calculate the energy of the signal in the time domain using `np.sum(x_t**2)`.

$$E = \sum_{n=-\infty}^{\infty} |x(n)|^2 \approx 46.3$$

2 Frequency Domain Signal

Using `numpy`, we apply the Fourier transform to the audio signal to obtain its frequency spectrum. Then we plot the signal in the frequency domain. To better examine the frequencies, we also present a zoomed-in version of the plot in figure (2).

```
import numpy.fft as FT
# FT centered around zero frequency
X_f = FT.fftshift(FT.fft(x_t))
# Define frequency domain
f = np.linspace(-f_s/2, f_s/2, len(X_f), endpoint=False)
```

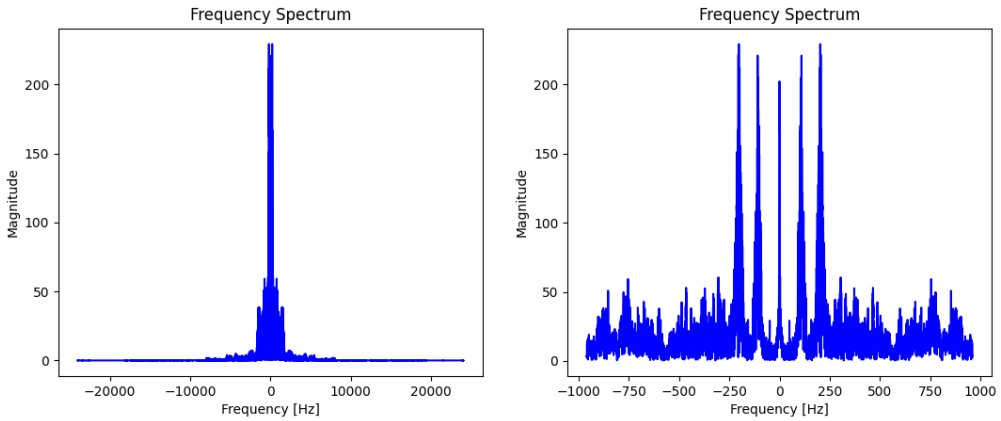


Figure 2: Frequency Domain Signal

If we calculate the energy of the signal in the frequency domain over a single period.

$$E = \sum_{k=0}^{N-1} |X(k)|^2 \approx 8533804.3$$

But Parseval's theorem states that the energy in both domains is equal! Now we must remember that a computer doesn't calculate CTFT, it calculates DTFT where Parseval's theorem has different statement.

Theorem (Parseval's Theorem for DTFT)**.**

$$\sum_{n=-\infty}^{\infty} |x(n)|^2 = \frac{1}{2\pi} \int_{-\pi}^{\pi} |X(e^{j\omega})|^2 d\omega$$

So if we calculate the *average power* of the frequency domain signal using `np.mean(np.abs(X_f)**2)` we can verify Parseval's theorem in this case.

$$\bar{P} = \frac{1}{N} \sum_{k=0}^{N-1} |X(k)|^2 \approx 46.3$$

And to find the exact frequency with maximum contribution to the signal we can use a very useful function in `numpy` called `argmax`.

```
index_max = np.argmax(np.abs(X_f))
f_max = f[index_max]
```

And we find that frequency to be ≈ 202.6 Hz.

3 Notch Filter

Given the Fourier transform of the signal we can implement an ideal notch filter by simply removing any desired frequency band. We implemented this notch filter with a bandwidth of 50, 100, and 200 Hz around the maximum frequency. Here, we plot the resulting frequency spectrum and time domain signal in each case.

```
df = f[1] - f[0] # The smallest frequency increment
bw = 50 # Notch filter bandwidth
window = bw//df
lower_bound = int(index_max - window//2)
upper_bound = int(index_max + window//2)

Notch_50Hz = X_f.copy()
Notch_50Hz[lower_bound:upper_bound] = 0
Notch_50Hz[len(X_f) - upper_bound:len(X_f) - lower_bound] = 0
```

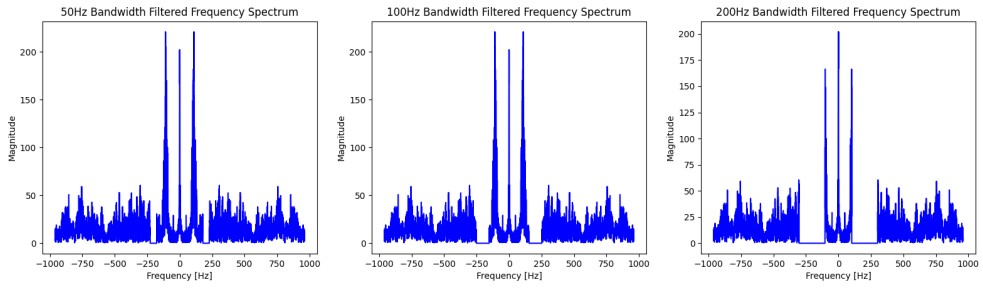


Figure 3: Frequency Spectrum after Notch Filtering

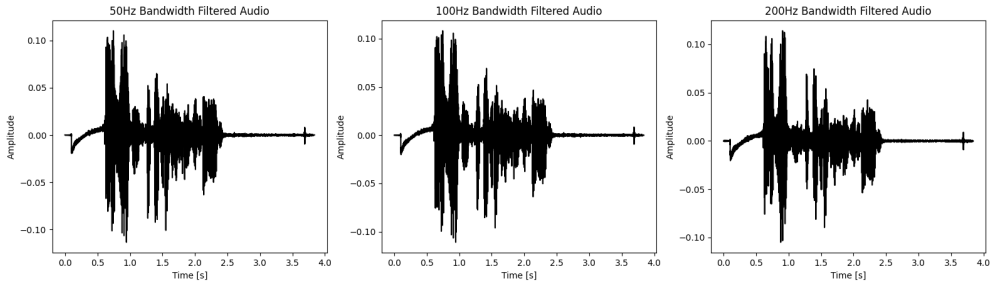


Figure 4: Time Domain Signal after Notch Filtering

We finally save the filtered signals using `soundfile`.

```
# Applying IFFT and saving as .wav file
x_t_50Hz_filtered = FT.ifft(FT.ifftshift(Notch_50Hz)).real
sf.write('Audio/Moaaz_50Hz_Filtered.wav', x_t_50Hz_filtered, f_s)
```